Supplement S2

Example 2: F1-score MIDOG Challenge

Table of contents

1 Introduction 1

2 Setup 2
- 2.1 Load Packages . . . 2
- 2.2 Functions . . . 2
- 2.3 Load Data . . . 2
- 2.4 Prepare Data . . . 2

3 Negative Controls 3

4 Objects per Domain 3

5 Part A: Overall analysis per model 4
- 5.1 Bootstrap . . . 4
- 5.2 Bayesian estimation . . . 5
  - 5.2.1 Fit Model . . . 5
  - 5.2.2 Get Posterior Draws . . . 5
- 5.3 Visualization . . . 6
  - 5.3.1 Figure 2 . . . 6

6 Part B: Stratified analysis by Domain 7
- 6.1 Bootstrap per domain . . . 8
- 6.2 Bayesian model with domain as fixed effect . . . 8
- 6.3 Visualization . . . 9
  - 6.3.1 Figure 3 . . . 9
  - 6.3.2 Supplement Figure S2.1 . . . 10

7 Checks 11
- 7.1 Checks Part A . . . 11
  - 7.1.1 Convergence diagnostics . . . 11
  - 7.1.2 Posterior predictive check . . . 12
- 7.2 Checks Part B . . . 18
  - 7.2.1 Convergence diagnostics . . . 18
  - 7.2.2 Posterior predictive check . . . 18

1 Introduction

Example 2 of the paper "Confidence intervals for the F1-score in medical object detection: comparing bootstrap and Bayesian multilevel methods".

Code and data: https://github.com/ndevln/f1-score-ci-object-detection (data: `S2_Data_MIDOG_Models.csv`).

This script estimates the F1-score and its 95% confidence/ credible interval on the MIDOG 2022 Challenge dataset (Aubreville et al. 2024, https://doi.org/10.1016/j.media.2022.102699): 3 baseline ML models evaluated on 100 images spanning 10 domains (0-213 mitotic figures per image).

Two methods, each for two estimands:

- Methods: bootstrap and Bayesian multilevel model (binomial + Poisson).
- Estimands: pooled (sum TP/FP/FN over the images, then compute the metric) and grouped (compute the metric per image, then average it).

The script has two parts:

- Part A - overall F1-score CI per model, both methods and both estimands.
- Part B - per-domain F1-score CI to probe domain generalization: bootstrap within each domain, and the Bayesian model with Domain as a fixed effect.

All intervals are 95% quantile intervals (QI, percentile interval), the point estimate is the median of the bootstrap or posterior draws. Images without objects (Case A, 4 of 100 images) are excluded and evaluated separately.

2 Setup

2.1 Load Packages

```
# Seed (Article Number of the MIDOG Paper)
set.seed(102699)

# Data manipulation and visualisation
library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)
library(ggdist)

# Bayesian packages
library(brms)       # Fits the multilevel models via Stan
library(tidybayes)  # median_qi()/epred_draws() helpers for posterior summaries
```

2.2 Functions

The F1-score is computed from the counts (paper eq. 3):

$$F_1 = \frac{2\,TP}{P + TP + FP}, \qquad P = TP + FN$$

Unlike the F1-score from the harmonic mean of precision and recall this formula is defined if TP = 0.

`format_result()` prints a point estimate with its interval as "0.65 (0.62 - 0.69)".

```
calc_F1_score <- function(TP, FP, P) {
  2 * TP / (P + TP + FP)
}

format_result <- function(value, .lower, .upper) {
  fmt <- scales::label_number(accuracy = 0.01, trim = F)
  glue::glue("{fmt(value)} ({fmt(.lower)} - {fmt(.upper)})")
}
```

2.3 Load Data

```
# One row per (model, slide): per-image confusion-matrix counts (TP/FP/FN) for each
# of the 3 baseline models, plus the Domain label (10 tissue/tumor types).
df_models <- readr::read_csv("S2_Data_MIDOG_Models.csv")
```

2.4 Prepare Data

```
df_models |>
  mutate(
    Domain = as.factor(Domain),   # treat the 10 domains as a categorical grouping
    GroundTruth = TP + FN,    # Sensitivity/recall trials: all true objects (Positives P)
    # Per-image point metrics
    Precision = pmax(0, (TP/(TP + FP)), na.rm = T), # Set to 0 if TP + FP = 0
    Recall = (TP/(TP + FN)),  # Cave: Not defined for images without true objects (P = 0)
```

```
    F1 = calc_F1_score(TP, FP, GroundTruth)
  ) |>
  # Exclude images without objects: F1-score is undefined
  filter(GroundTruth > 0) -> df_models_prep
```

3 Negative Controls

Images without objects (P = 0) do not contribute to the F1-score (no TP or FN) and are reported separately.

```
df_models |>
  filter(TP + FN == 0) |>
  group_by(model) |>
  summarise(
    `n images`          = n(),
    `n correct (no FP)` = sum(FP == 0),
    `FP total`          = sum(FP),
    `FP per image`      = paste(FP, collapse = ", ")
  )
```

model	n images	n correct (no FP)	FP total	FP per image
Model 1	4	1	5	3, 1, 1, 0
Model 2	4	2	6	0, 1, 5, 0
Model 3	4	1	14	10, 0, 2, 2

4 Objects per Domain

Images with objects per domain (the ground truth is the same for all models): objects per image P (median and range), images with at most 5 objects, objects in total, and the share of all images and objects. Images with few objects make up a relevant share of the images, but only a small share of the objects, so they contribute little to the pooled estimand (all objects weighted equally) and fully to the grouped estimand (all images weighted equally).

```
df_models_prep |>
  filter(model == "Model 1") |>
  mutate(Domain = as.character(Domain)) -> df_objects

# Domains and a total row
bind_rows(df_objects, mutate(df_objects, Domain = "Total")) |>
  mutate(Domain = factor(Domain, levels = c(1:10, "Total"))) |>
  group_by(Domain) |>
  summarise(
    Images = n(),
    `P median (range)` = sprintf("%g (%d-%d)", median(GroundTruth),
                                 min(GroundTruth), max(GroundTruth)),
    `P <= 5` = sum(GroundTruth <= 5),
    Objects = sum(GroundTruth)
  ) |>
  mutate(
    `% images`  = round(100 * Images / nrow(df_objects), 1),
    `% objects` = round(100 * Objects / sum(df_objects$GroundTruth), 1)
  )
```

Domain	Images	P median (range)	P <= 5	Objects	% images	% objects
1	10	10 (1-69)	3	209	10.4	6.0
2	7	4 (4-9)	5	37	7.3	1.1
3	10	53.5 (4-85)	1	471	10.4	13.6
4	10	19 (3-59)	2	213	10.4	6.1
5	10	15.5 (2-43)	2	174	10.4	5.0
6	9	2 (1-80)	6	148	9.4	4.3
7	10	41 (16-156)	0	497	10.4	14.3
8	10	30 (5-202)	1	488	10.4	14.0

Domain	Images	P median (range)	P <= 5	Objects	% images	% objects
9	10	33.5 (11-213)	0	580	10.4	16.7
10	10	63.5 (12-121)	0	657	10.4	18.9
Total	96	27 (1-213)	20	3474	100.0	100.0

5 Part A: Overall analysis per model

5.1 Bootstrap

The images are resampled with replacement (K = 10,000 resamples, >= 10,000 recommended for a stable 95% CI). Both estimands are computed from the same resampled images:

- Pooled: sum TP, FP and FN over all resampled images first, then compute the metrics. Images are weighted by their object count (P), so high-count images dominate.
- Grouped: average the per-image metrics, so every image carries equal weight regardless of its object count (see paper "Grouped averaging").

```
K = 10000

df_models_prep |>
  select(model, TP:FN, Precision:F1) |>
  # One bootstrap "study" per .draw
  cross_join(tibble(.draw = 1:K)) |>
  group_by(model, .draw) |>
  # Resample images with replacement up to the number of observations
  slice_sample(prop = 1, replace = TRUE) |>
  summarise(
    # POOLED
    Precision_pooled = pmax(0, sum(TP) / sum(TP + FP), na.rm = T), # 0 if TP + FP = 0
    Recall_pooled = sum(TP) / sum(TP + FN),
    F1_pooled = calc_F1_score(sum(TP), sum(FP), sum(TP + FN)),
    # GROUPED
    Precision_grouped = mean(Precision),
    Recall_grouped = mean(Recall),
    F1_grouped = mean(F1)
  ) |>
  # Long format: one row per (draw, metric, averaging)
  pivot_longer(matches("_(pooled|grouped)$"), names_to = c("name", "averaging"),
               names_sep = "_") |>
  mutate(method = "Bootstrap") -> draws_models_boot

draws_models_boot |>
  group_by(method, model, averaging, name) |>
  median_qi(value) |>
  arrange(model, name) |>
  mutate(format = format_result(value, .lower, .upper)) |>
  select(method, model, averaging, name, format)
```

method	model	averaging	name	format
Bootstrap	Model 1	grouped	F1	0.56 (0.52 - 0.61)
Bootstrap	Model 1	pooled	F1	0.65 (0.62 - 0.68)
Bootstrap	Model 1	grouped	Precision	0.67 (0.62 - 0.73)
Bootstrap	Model 1	pooled	Precision	0.77 (0.73 - 0.81)
Bootstrap	Model 1	grouped	Recall	0.55 (0.50 - 0.59)
Bootstrap	Model 1	pooled	Recall	0.57 (0.53 - 0.61)
Bootstrap	Model 2	grouped	F1	0.64 (0.60 - 0.68)
Bootstrap	Model 2	pooled	F1	0.71 (0.68 - 0.74)
Bootstrap	Model 2	grouped	Precision	0.69 (0.64 - 0.74)
Bootstrap	Model 2	pooled	Precision	0.77 (0.73 - 0.81)
Bootstrap	Model 2	grouped	Recall	0.65 (0.61 - 0.69)
Bootstrap	Model 2	pooled	Recall	0.66 (0.62 - 0.70)
Bootstrap	Model 3	grouped	F1	0.45 (0.39 - 0.51)
Bootstrap	Model 3	pooled	F1	0.51 (0.44 - 0.58)

method	model	averaging	name	format
Bootstrap	Model 3	grouped	Precision	0.67 (0.60 - 0.73)
Bootstrap	Model 3	pooled	Precision	0.76 (0.71 - 0.80)
Bootstrap	Model 3	grouped	Recall	0.42 (0.36 - 0.48)
Bootstrap	Model 3	pooled	Recall	0.39 (0.32 - 0.47)

5.2 Bayesian estimation

5.2.1 Fit Model

```
# Split the data into one dataset per model
data_list <- df_models_prep |>
  split(~ model)

# Binomial + Poisson model
# Sensitivity Model (Recall): TP successes out of GroundTruth (= TP + FN) trials
bf_tp <- bf(TP | trials(GroundTruth) ~ 1 + (1|Slide), family = binomial())
# False positive model: FP count per image, log link
bf_fp <- bf(FP ~ 1 + (1|Slide), family = poisson())
# (1|Slide): per-image random effect in each sub-model.
# Precision follows from both parts: recall * P / (recall * P + FP).

# Fit the model for every dataset (model) at once; default brms weakly-informative
# priors; 4 chains, 2000 warmup of 10,000 iterations.
F1_model_multi <- brm_multiple(
  formula = bf_tp + bf_fp,
  data = data_list,
  chains = 4,
  warmup = 2000,
  iter = 10000,
  seed = 102699,
  refresh = 0, # No sampling progress in the output for PDF
  combine = FALSE # Keep model results seperate
)
```

5.2.2 Get Posterior Draws

The marginal estimates refer to the population of images that the observed images represent (CAVE: if images are a convenience sample). For every posterior draw:

1. Expected TP and FP of every observed image with its fitted image effect (`epred_images()`, trials = 1 gives the recall, expected TP = recall × P).
2. Bayesian bootstrap weights over the images, Dirichlet(1, …, 1) via `rexp()`, new for every draw.
3. Pooled and grouped metrics with these weights (`calc_marginal_metrics()`).

Expected counts are never 0/0, so there are no continuity problems.

```
epred_images <- function(brms_model, newdata) {
  epred_draws(brms_model, newdata = newdata |> mutate(GroundTruth = 1)) |>
    ungroup() |>
    select(.draw, any_of("Domain"), Slide, P, .category, .epred) |>
    pivot_wider(names_from = .category, values_from = .epred) |>
    mutate(
      Recall = TP,               # recall of the image
      TP = Recall * P,           # expected TP of the image
      Precision = TP / (TP + FP) # expected TP / (expected TP + expected FP)
    ) |>
    # Bayesian bootstrap weights, new for every draw
    group_by(.draw) |>
    mutate(weight = rexp(n())) |>
    ungroup()
}

calc_marginal_metrics <- function(df_images) {
```

```
  df_images |>
    summarise(
      # POOLED
      Precision_pooled = sum(weight * TP) / sum(weight * (TP + FP)),
      Recall_pooled = sum(weight * TP) / sum(weight * P),
      F1_pooled = calc_F1_score(sum(weight * TP), sum(weight * FP), sum(weight * P)),
      # GROUPED
      Precision_grouped = weighted.mean(Precision, weight),
      Recall_grouped = weighted.mean(Recall, weight),
      F1_grouped = weighted.mean(calc_F1_score(TP, FP, P), weight),
      .groups = "drop"
    ) |>
    # Long format: one row per (draw, metric, averaging)
    pivot_longer(matches("_(pooled|grouped)$"), names_to = c("name", "averaging"),
                 names_sep = "_")
}

# Per ML model: the observed images with their object count P
draws_models_bayesian <- purrr::map2_dfr(F1_model_multi, data_list, \(x, y)
  epred_images(x, newdata = y |> distinct(Slide, P = GroundTruth)) |>
    group_by(.draw) |>
    calc_marginal_metrics() |>
    mutate(model = unique(y$model))
) |>
  mutate(method = "Bayesian")
```

Point estimate (posterior median) and 95% QI per model, estimand and metric:

```
draws_models_bayesian |>
  group_by(method, model, averaging, name) |>
  median_qi(value) |>
  arrange(model, name) |>
  mutate(format = format_result(value, .lower, .upper)) |>
  select(method, model, averaging, name, format)
```

method	model	averaging	name	format
Bayesian	Model 1	grouped	F1	0.57 (0.53 - 0.61)
Bayesian	Model 1	pooled	F1	0.65 (0.62 - 0.69)
Bayesian	Model 1	grouped	Precision	0.66 (0.61 - 0.71)
Bayesian	Model 1	pooled	Precision	0.77 (0.72 - 0.81)
Bayesian	Model 1	grouped	Recall	0.55 (0.51 - 0.59)
Bayesian	Model 1	pooled	Recall	0.57 (0.53 - 0.61)
Bayesian	Model 2	grouped	F1	0.64 (0.60 - 0.67)
Bayesian	Model 2	pooled	F1	0.71 (0.68 - 0.74)
Bayesian	Model 2	grouped	Precision	0.68 (0.63 - 0.72)
Bayesian	Model 2	pooled	Precision	0.77 (0.73 - 0.81)
Bayesian	Model 2	grouped	Recall	0.65 (0.61 - 0.69)
Bayesian	Model 2	pooled	Recall	0.66 (0.62 - 0.70)
Bayesian	Model 3	grouped	F1	0.44 (0.39 - 0.50)
Bayesian	Model 3	pooled	F1	0.51 (0.44 - 0.58)
Bayesian	Model 3	grouped	Precision	0.67 (0.60 - 0.73)
Bayesian	Model 3	pooled	Precision	0.76 (0.70 - 0.81)
Bayesian	Model 3	grouped	Recall	0.41 (0.35 - 0.47)
Bayesian	Model 3	pooled	Recall	0.39 (0.32 - 0.47)

5.3 Visualization

5.3.1 Figure 2

```
bind_rows(draws_models_boot, draws_models_bayesian) |>
  filter(name == "F1") |>
  ggplot(aes(y = interaction(method, averaging), x = value)) +
```

```r
# One distribution per method and estimand, with median point + 95% QI.
stat_sample_slabinterval(aes(fill = method), show.legend = F,
                         point_interval = "median_qi", .width = 0.95,
                         slab_alpha = 0.7, normalize = "xy") +
# Observed per-image F1 as points below; point size = number of objects (GroundTruth)
ggbeeswarm::geom_beeswarm(data = df_models_prep |> arrange(desc(GroundTruth)),
                          aes(x = F1, y = 0.8, size = GroundTruth),
                          alpha = 0.7, cex = 4, side = -1L, method = "compactswarm") +
facet_grid(model ~ .) +   # one panel per ML model
ggokabeito::scale_fill_okabe_ito() +   # colour-blind-safe palette
ggsurveillance::scale_y_discrete_reverse(guide = legendry::guide_axis_nested(), name = NULL) +
scale_x_continuous(name = "F1-score", breaks = (0:10) / 10) +
scale_size(name = "Slides (number of objects on each slide)", range = c(0.2, 2),
           breaks = c(1, 50, 100, 150)) +
theme_bw() +
ggsurveillance::theme_mod_legend_top() +
ggsurveillance::theme_mod_remove_minor_grid_x()
```

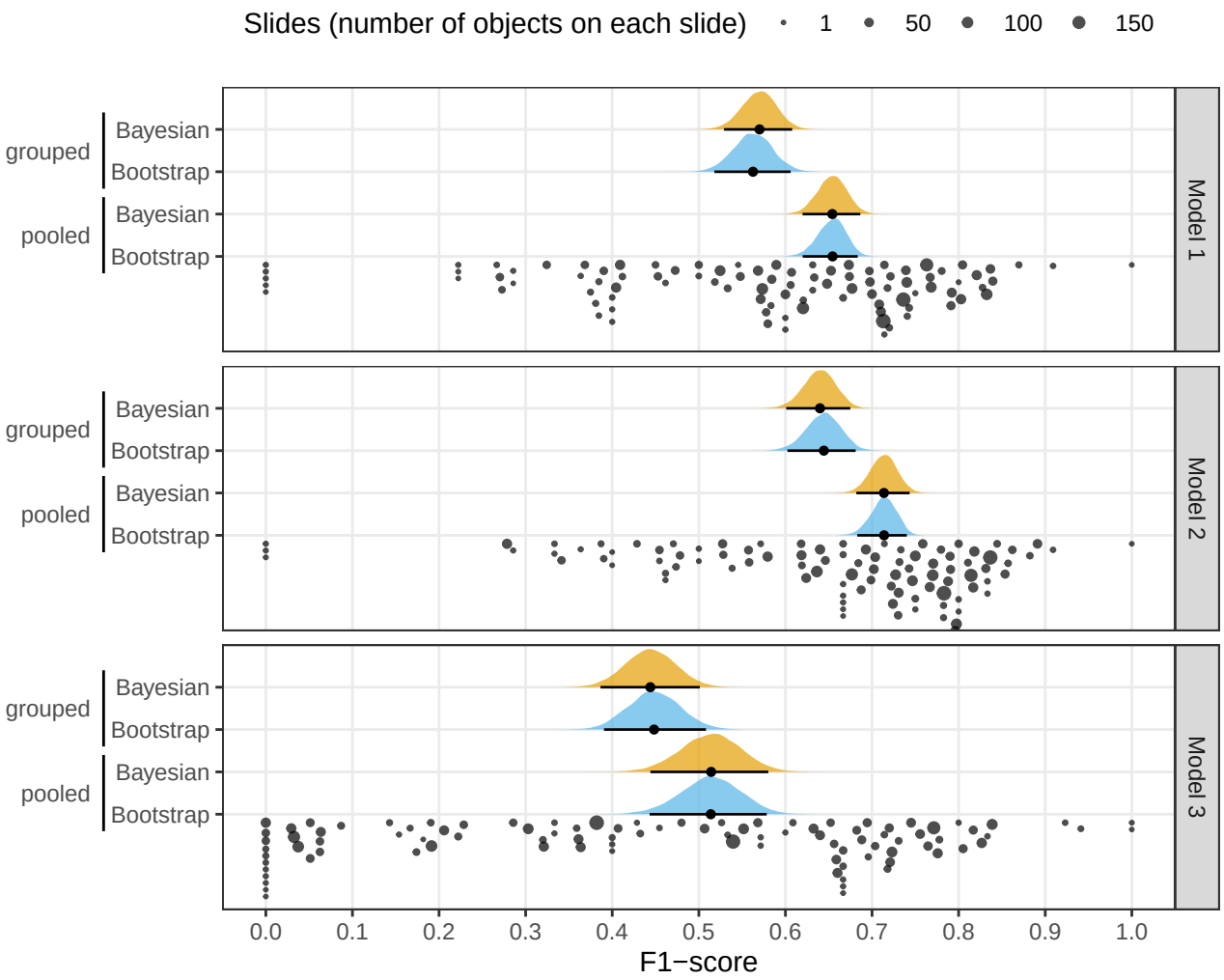


Figure 2: F1-score distributions of both methods and both estimands per model.

```r
ggsave("Figure_2.pdf", width = 8, height = 5.5)
```

6 Part B: Stratified analysis by Domain

Domain generalization diagnostic. The MIDOG challenge cares whether a model performs consistently across the 10 domains. Both methods and both estimands are computed per domain: the bootstrap resamples the images within each domain, the Bayesian model has Domain as a fixed effect and the Bayesian bootstrap weights act within each domain.

6.1 Bootstrap per domain

```
df_models_prep |>
  select(model, Domain, TP:FN, Precision:F1) |>
  cross_join(tibble(.draw = 1:K)) |>
  # Stratify by Domain: resampling happens within each domain
  group_by(model, Domain, .draw) |>
  # Resample with replacement up to the number of observations
  slice_sample(prop = 1, replace = TRUE) |>
  summarise(
    # POOLED
    Precision_pooled = pmax(0, sum(TP) / sum(TP + FP), na.rm = T), # 0 if TP + FP = 0
    Recall_pooled = sum(TP) / sum(TP + FN),
    F1_pooled = calc_F1_score(sum(TP), sum(FP), sum(TP + FN)),
    # GROUPED
    Precision_grouped = mean(Precision),
    Recall_grouped = mean(Recall),
    F1_grouped = mean(F1),
    .groups = "drop"
  ) |>
  # Long format: one row per (domain, draw, metric, averaging)
  pivot_longer(matches("_(pooled|grouped)$"), names_to = c("name", "averaging"),
               names_sep = "_") |>
  mutate(method = "Bootstrap\n(stratified)") -> draws_models_boot_domain
```

6.2 Bayesian model with domain as fixed effect

```
# Re-split per model (as before) and refit with Domain added as a fixed effect
# This is the Bayesian analogue of the domain-stratified bootstrap.
data_list <- df_models_prep |>
  split(~ model)

# Binomial + Poisson model as above, with Domain as fixed effect
# Sensitivity Model (Recall): Domain fixed effect + per-image random effect
bf_tp_d <- bf(TP | trials(GroundTruth) ~ Domain + (1|Slide), family = binomial())
# False positive model: same structure
bf_fp_d <- bf(FP ~ Domain + (1|Slide), family = poisson())

# Fit per model; same sampler settings as the non-stratified model above.
F1_model_domain_multi <- brm_multiple(
  formula = bf_tp_d + bf_fp_d,
  data = data_list,
  chains = 4,
  warmup = 2000,
  iter = 10000,
  seed = 102699,
  refresh = 0,
  combine = FALSE
)
```

Marginal estimates per domain as in Part A, with `epred_images()` and `calc_marginal_metrics()` per domain and posterior draw.

```
draws_models_bayesian_domain <- purrr::map2_dfr(F1_model_domain_multi, data_list, \(x, y)
  epred_images(x, newdata = y |> distinct(Domain, Slide, P = GroundTruth)) |>
    group_by(Domain, .draw) |>
    calc_marginal_metrics() |>
    mutate(model = unique(y$model))
) |>
  mutate(method = "Bayesian\n(fixed effect)")
```

6.3 Visualization

6.3.1 Figure 3

```
bind_rows(draws_models_boot_domain, draws_models_bayesian_domain) |>
  filter(name == "F1") -> draws_models_domain_F1

draws_models_domain_F1 |>
  filter(averaging == "pooled") |>
  ggplot(aes(y = method, x = value)) +
  # One coloured density slab per domain (adjust = 3 smooths the discrete bootstrap).
  stat_slab(aes(fill = Domain), show.legend = F, slab_alpha = 0.6, normalize = "xy",
            adjust = 3, linewidth = 0.2, color = "black") +
  # Observed per-image F1 below, grouped on the y-axis by domain (1 - Domain/10) and
  # coloured by domain; point size = number of objects on the slide.
  ggbeeswarm::geom_beeswarm(data = df_models_prep |> arrange(desc(GroundTruth)),
                            aes(x = F1, y = 1 - as.numeric(Domain) / 10,
                                size = GroundTruth, color = Domain),
                            alpha = 0.7, cex = 2, side = -1L, dodge.width = 0.1,
                            method = "compactswarm") +
  facet_grid(model ~ .) +   # one row of panels per ML model
  ggsci::scale_fill_d3(guide = NULL) +    # 10-colour categorical palette for domains
  ggsci::scale_color_d3(guide = NULL) +
  ggsurveillance::scale_y_discrete_reverse(name = NULL) +
  scale_x_continuous(name = "F1-score", breaks = (0:10) / 10) +
  scale_size(name = "Slides (number of objects on each slide)", range = c(0.2, 2),
             breaks = c(1, 50, 100, 150)) +
  theme_bw() +
  ggsurveillance::theme_mod_legend_top() +
  ggsurveillance::theme_mod_remove_minor_grid_x() -> fig_domain

fig_domain
```

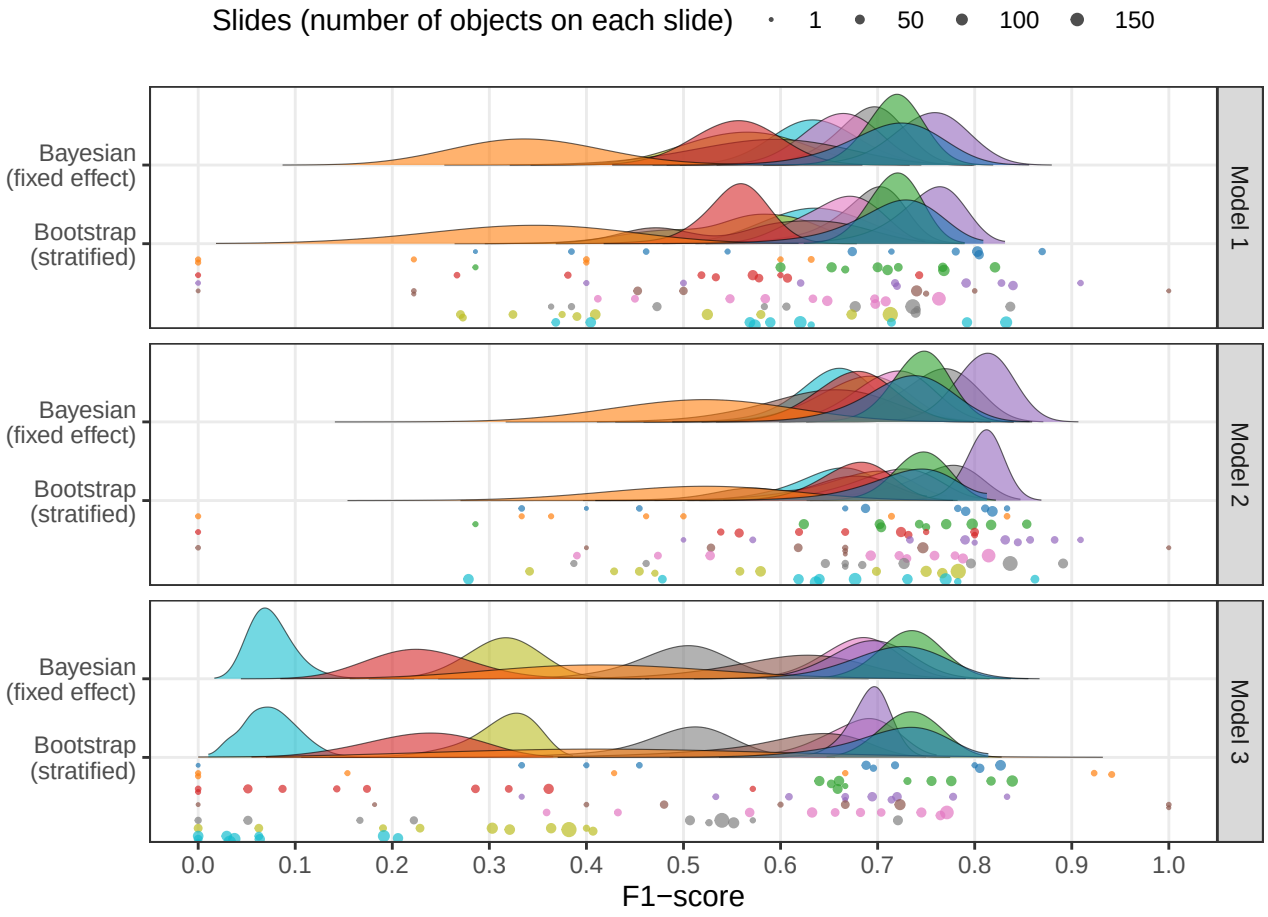
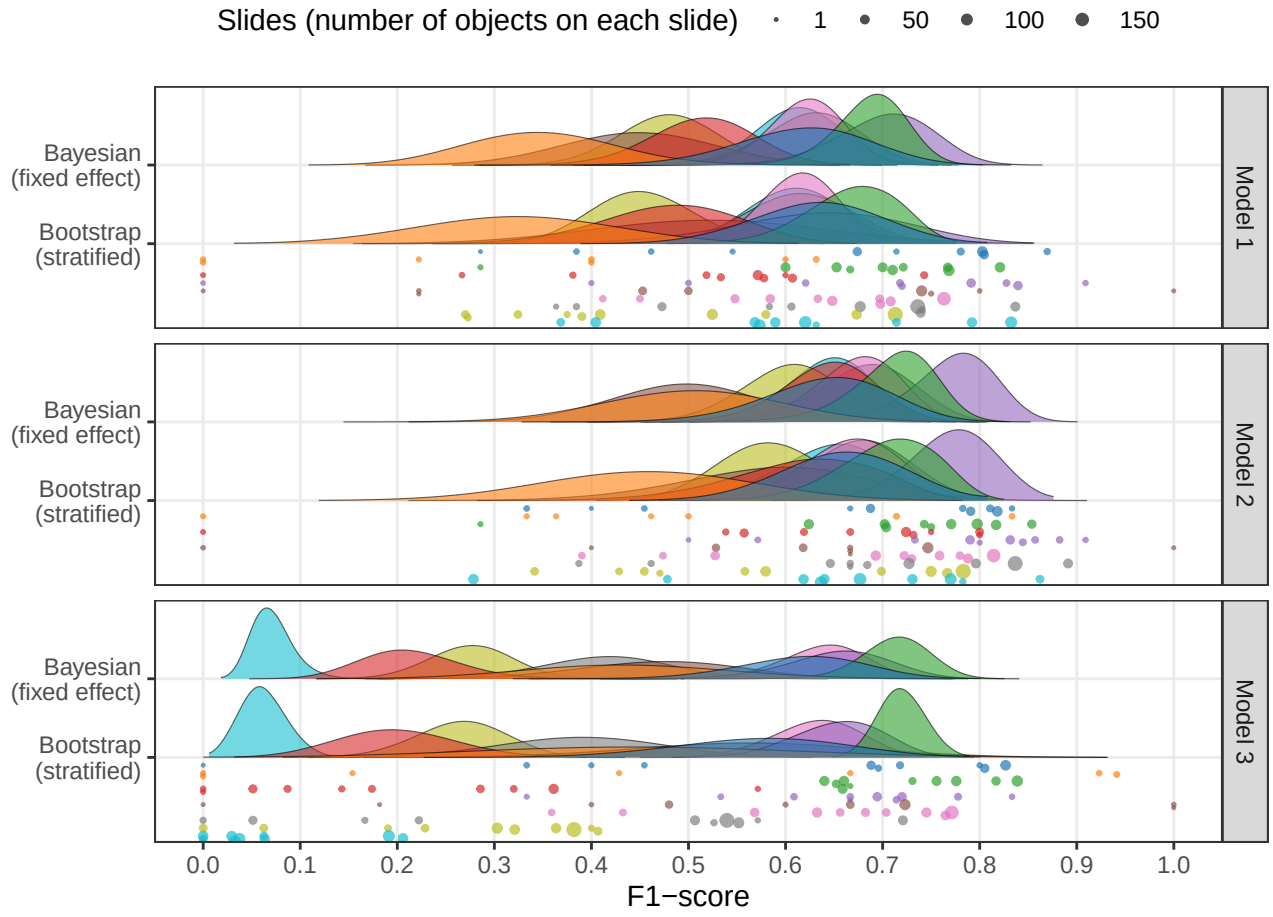


Figure 3: F1-score distributions per domain, both methods, pooled estimand.

```
ggsave("Figure_3.pdf", width = 7, height = 6)
```

6.3.2 Supplement Figure S2.1

```
# Figure 3 with its data replaced by the grouped draws
fig_domain + filter(draws_models_domain_F1, averaging == "grouped")
```



Supplement Figure S2.1: F1-score distributions per domain, both methods, grouped estimand.

7 Checks

7.1 Checks Part A

7.1.1 Convergence diagnostics

```
# Convergence of the fits of Part A, per ML model, over all parameters
purrr::map2_dfr(F1_model_multi, names(data_list), \(x, y) {
  df_summary <- posterior::summarise_draws(as_draws_array(x), "rhat", "ess_bulk", "ess_tail")
  tibble(
    model        = y,
    max_rhat     = max(df_summary$rhat, na.rm = TRUE),
    min_ess_bulk = min(df_summary$ess_bulk, na.rm = TRUE),
    min_ess_tail = min(df_summary$ess_tail, na.rm = TRUE),
    divergences  = sum(subset(nuts_params(x), Parameter == "divergent__")$Value)
  )
})
```

model	max_rhat	min_ess_bulk	min_ess_tail	divergences
Model 1	1.000786	6197.403	11625.768	0
Model 2	1.001632	5715.207	12083.742	0
Model 3	1.000903	3747.097	7312.211	0

```
# Part A per population-level parameter (intercepts and SD of the per-image effects)
purrr::map2_dfr(F1_model_multi, names(data_list), \(x, y)
  posterior::summarise_draws(as_draws_array(x, variable = "^(b|sd)_", regex = TRUE),
```

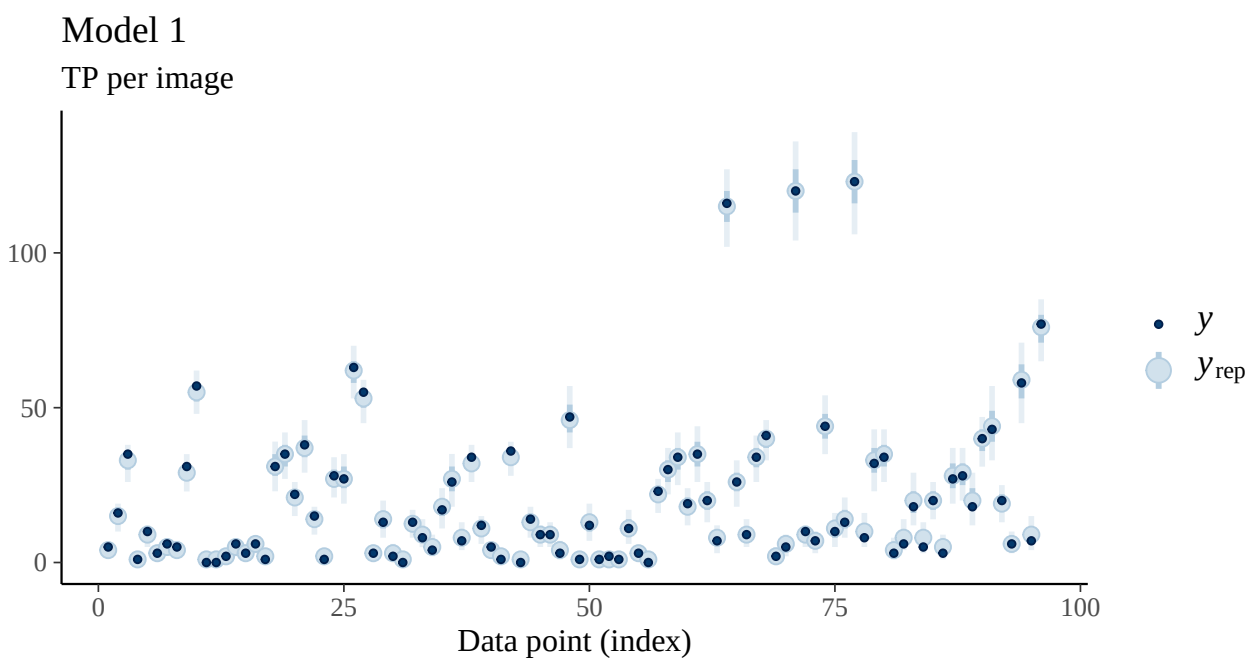
```
                          "rhat", "ess_bulk", "ess_tail") |>
    mutate(model = y, .before = 1)
)
```

model	variable	rhat	ess_bulk	ess_tail
Model 1	b_TP_Intercept	1.000786	6344.698	11943.025
Model 1	b_FP_Intercept	1.000250	7499.182	14188.674
Model 1	sd_Slide__TP_Intercept	1.000274	9329.243	17124.272
Model 1	sd_Slide__FP_Intercept	1.000144	7677.005	14947.875
Model 2	b_TP_Intercept	1.000153	7212.738	13436.249
Model 2	b_FP_Intercept	1.000368	7411.966	14126.143
Model 2	sd_Slide__TP_Intercept	1.000436	9483.743	14965.020
Model 2	sd_Slide__FP_Intercept	1.000625	7431.819	14192.985
Model 3	b_TP_Intercept	1.000283	3747.097	7312.211
Model 3	b_FP_Intercept	1.000903	7867.353	13599.235
Model 3	sd_Slide__TP_Intercept	1.000494	6161.100	10689.297
Model 3	sd_Slide__FP_Intercept	1.000316	6744.278	11103.497

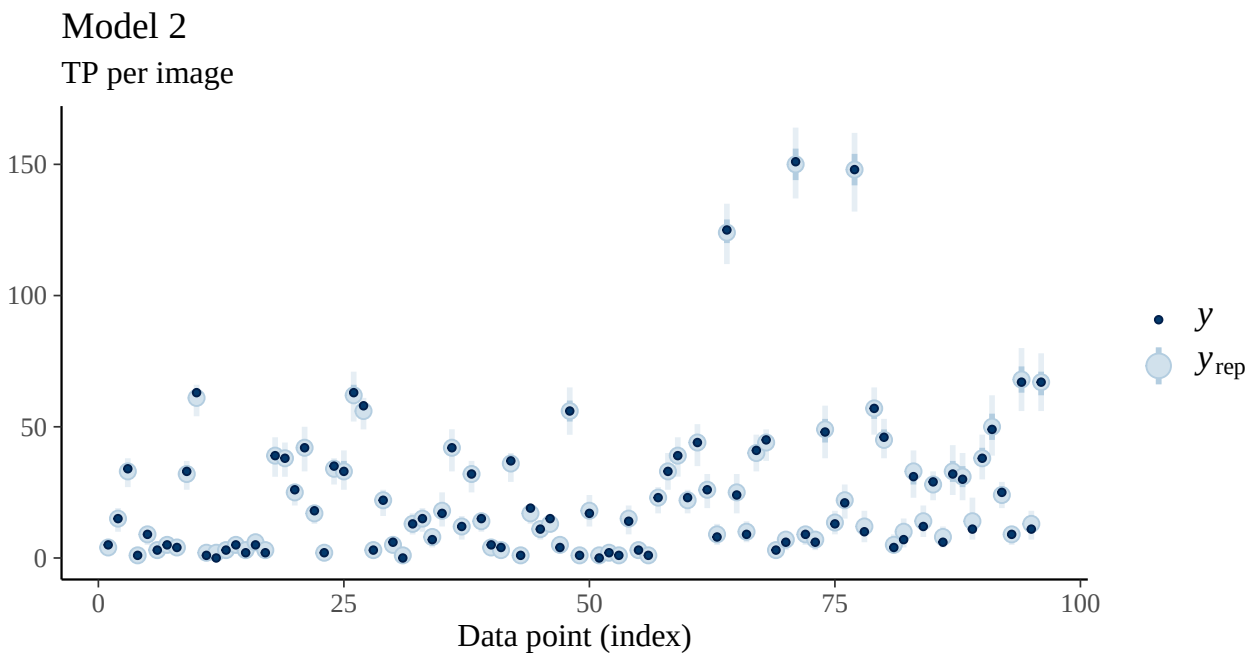
7.1.2 Posterior predictive check

```
# TP: observed TP per image (dot) with the 50% / 90% predictive interval
purrr::map2(F1_model_multi, names(data_list), \(x, y)
  pp_check(x, resp = "TP", type = "intervals", ndraws = 1000) + ggtitle(y, "TP per image")
)
```

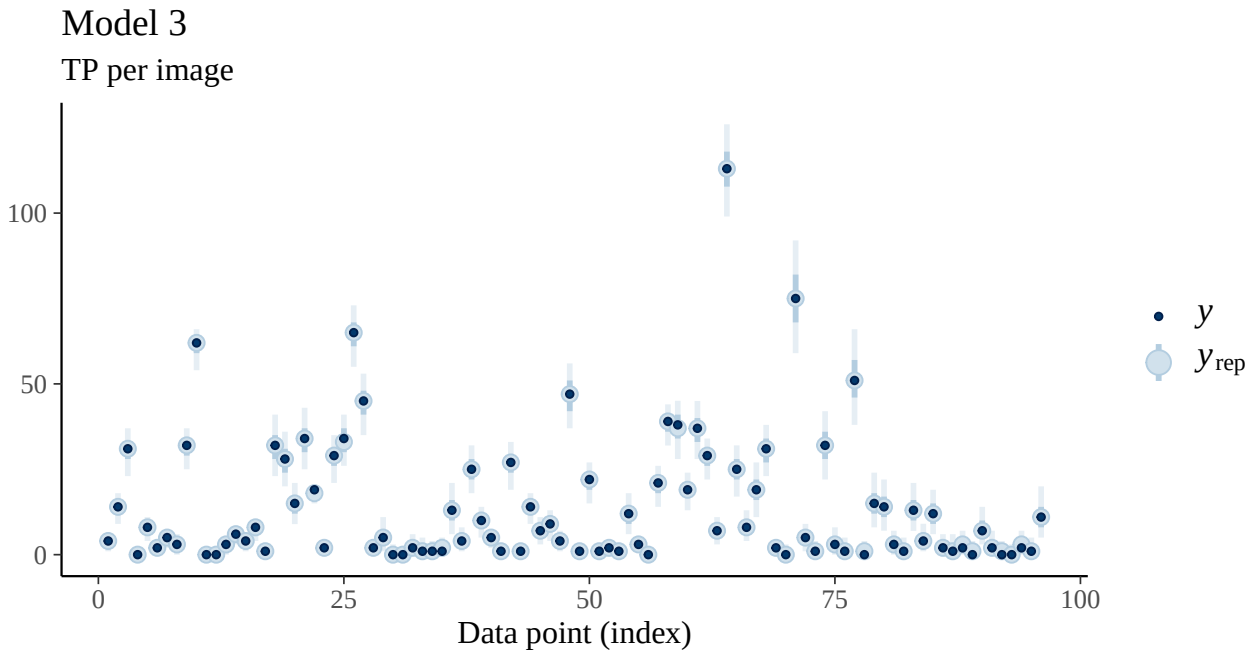
```
[[1]]
```




```
[[2]]
```

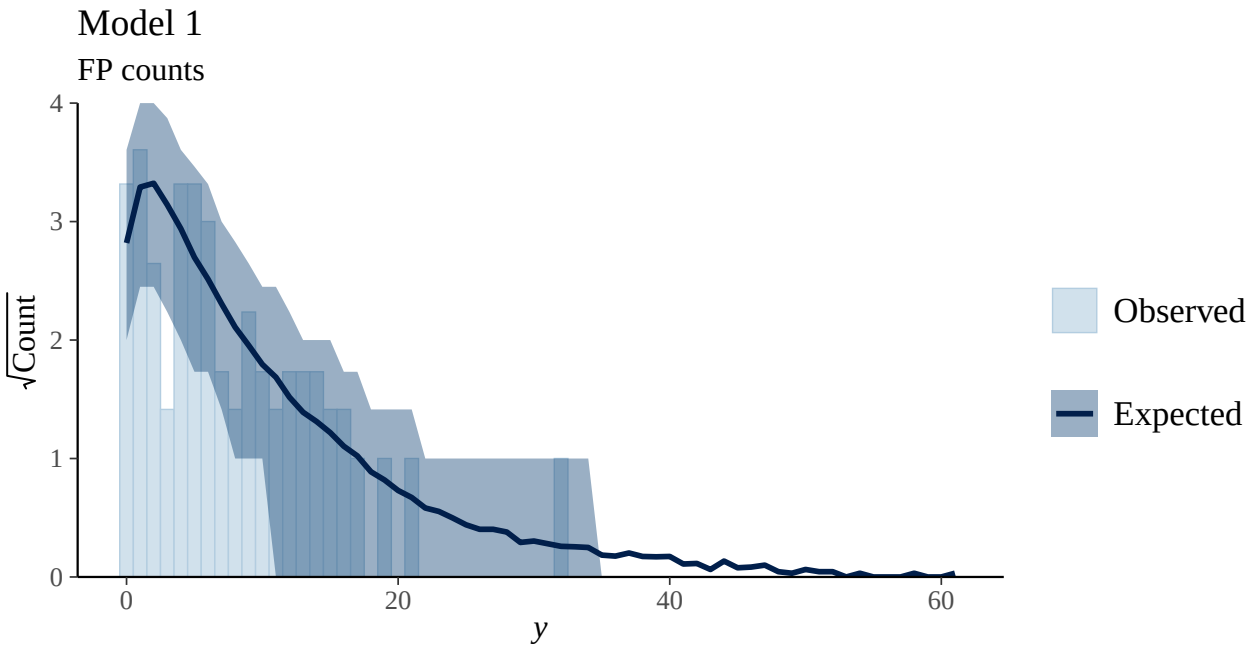



```
[[3]]
```

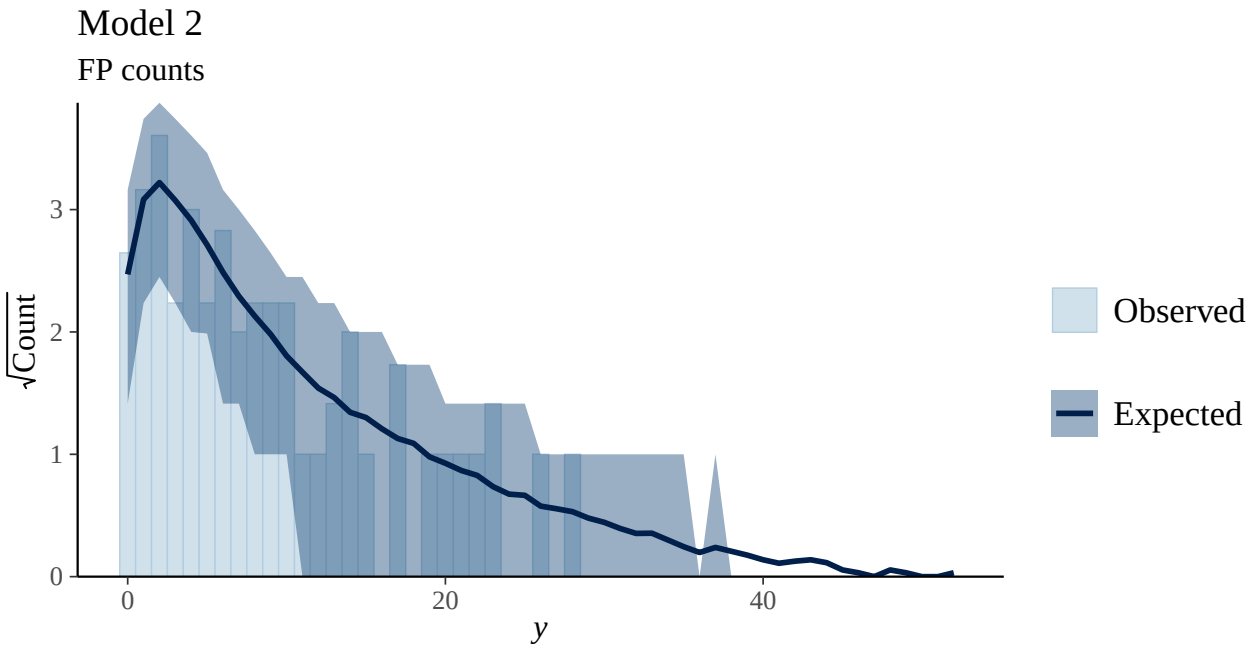



```
# FP: rootogram of the FP counts (observed bars vs. expected frequencies)
purrr::map2(F1_model_multi, names(data_list), \(x, y)
  pp_check(x, resp = "FP", type = "rootogram", ndraws = 1000) + ggtitle(y, "FP counts")
)
```

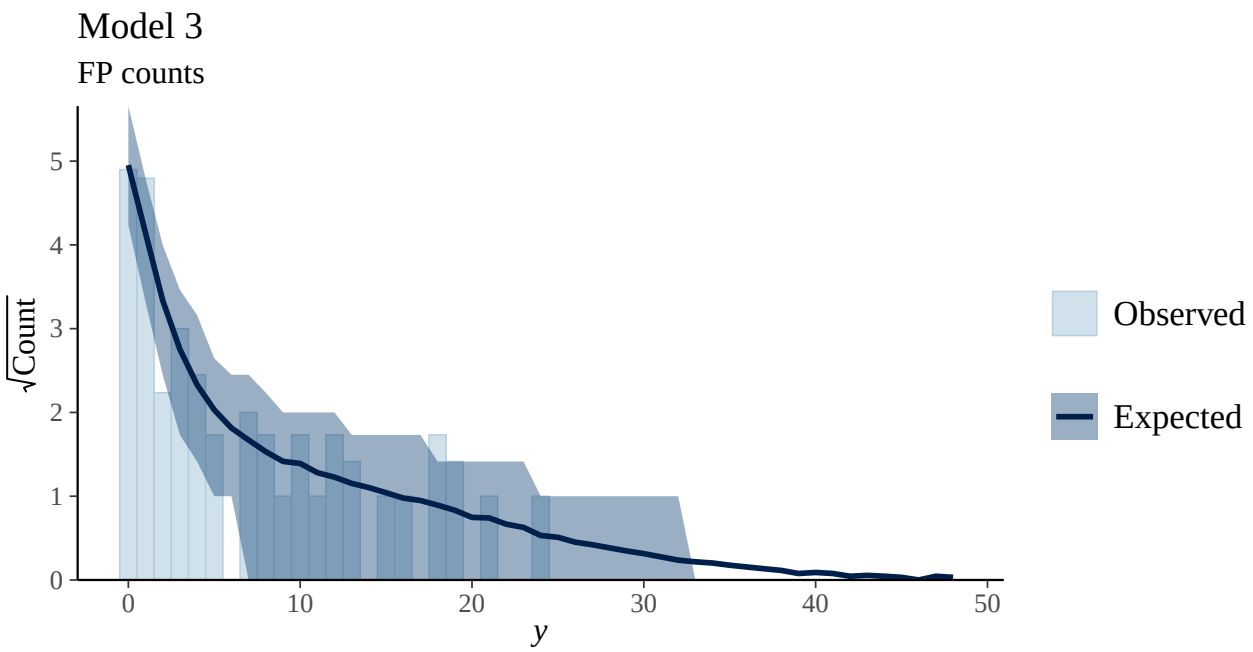
```
[[1]]
```


Model 1
FP counts
√Count
y
Observed
Expected

[[2]]

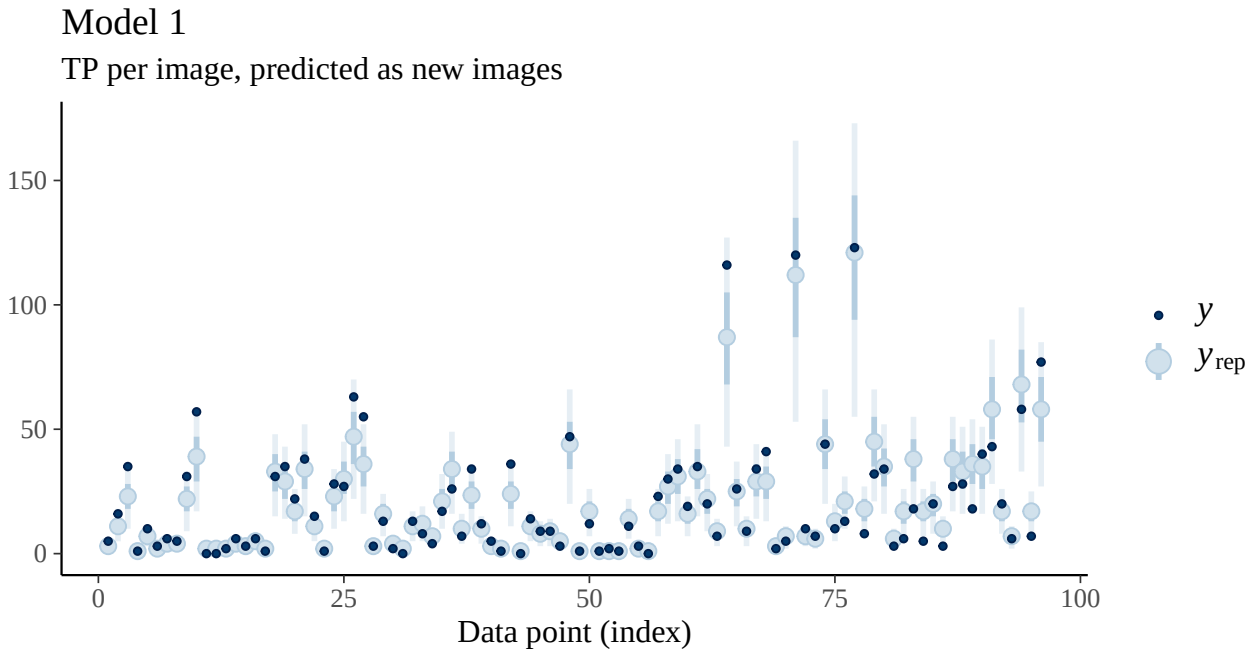

Model 2
FP counts
√Count
y
Observed
Expected

[[3]]

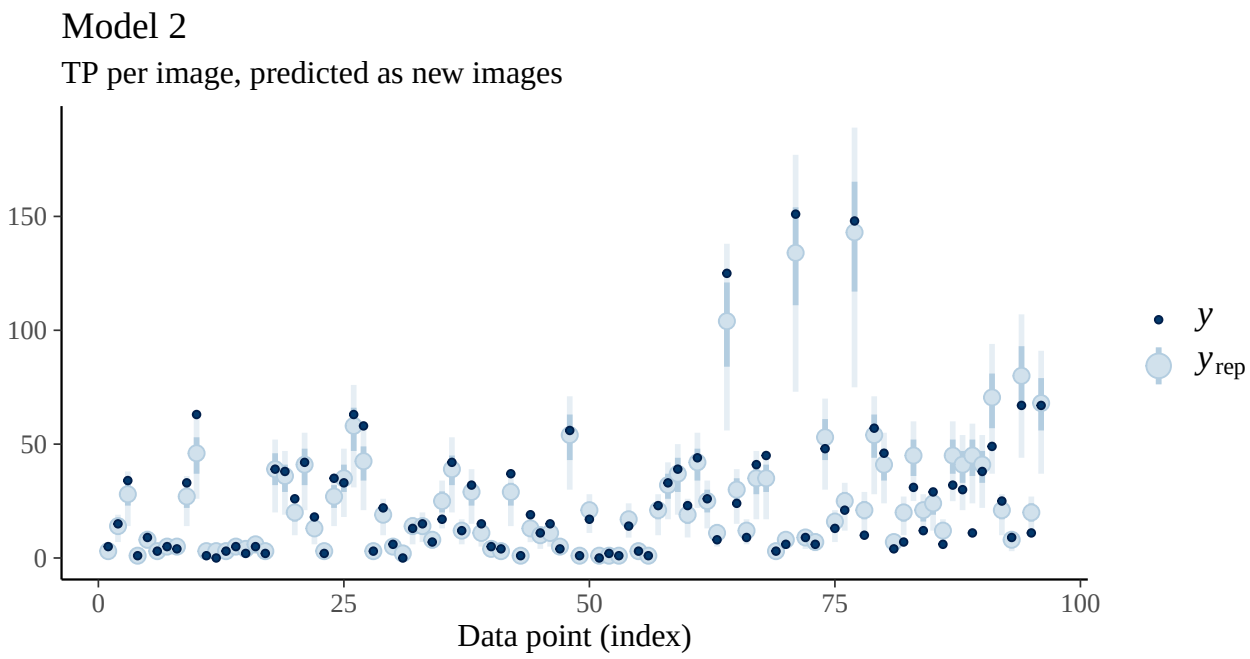



```
# TP per image, predicted as new images
purrr::map2(F1_model_multi, data_list, \(x, y)
  pp_check(x, resp = "TP", type = "intervals", ndraws = 1000,
           newdata = y |> mutate(Slide = paste0("new_", Slide)),
           allow_new_levels = TRUE, sample_new_levels = "gaussian") +
    ggtitle(unique(y$model), "TP per image, predicted as new images")
)
```

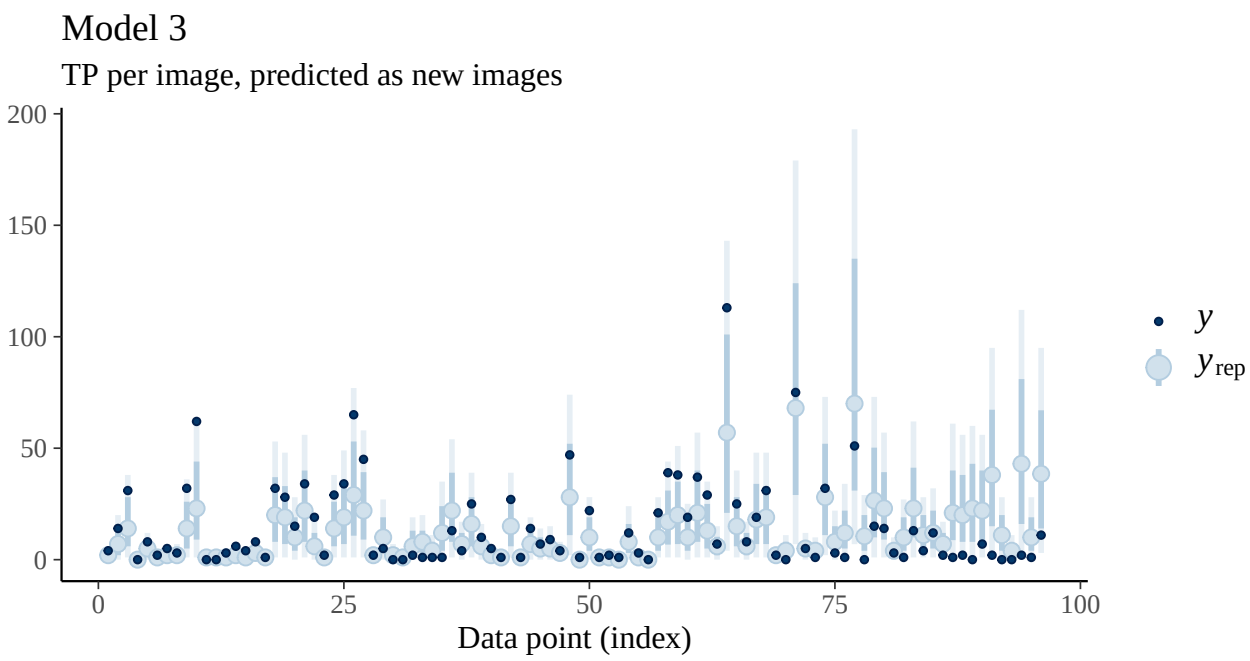
```
[[1]]
```




```
[[2]]
```

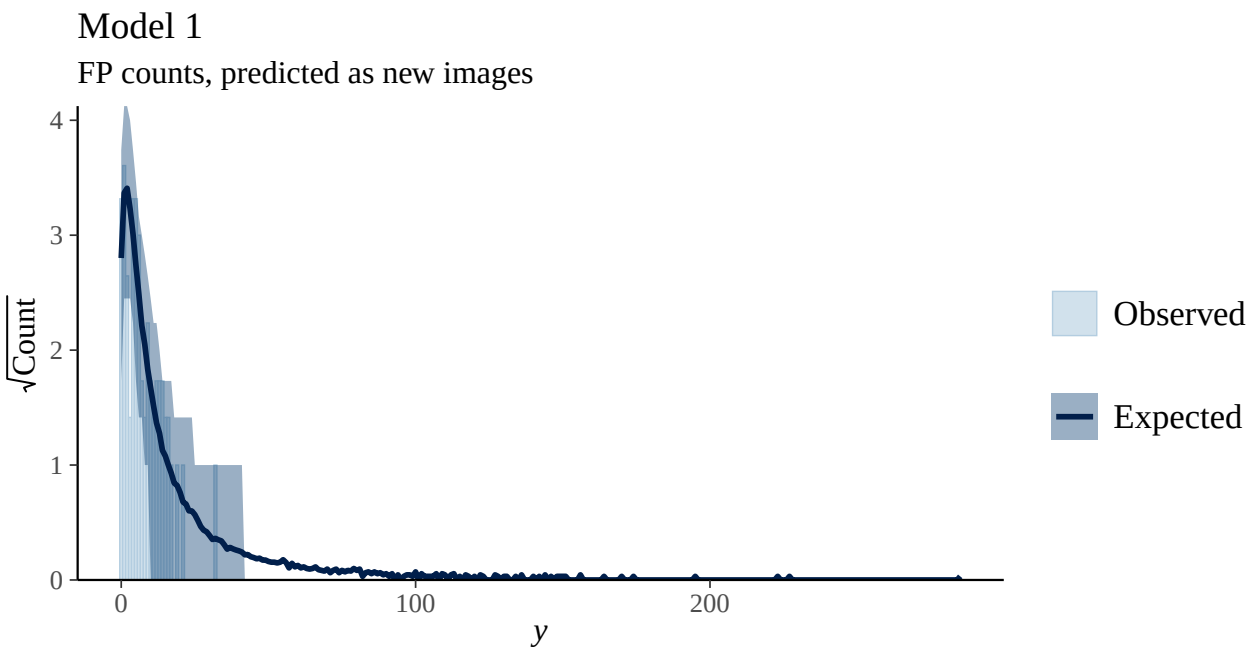



```
[[3]]
```

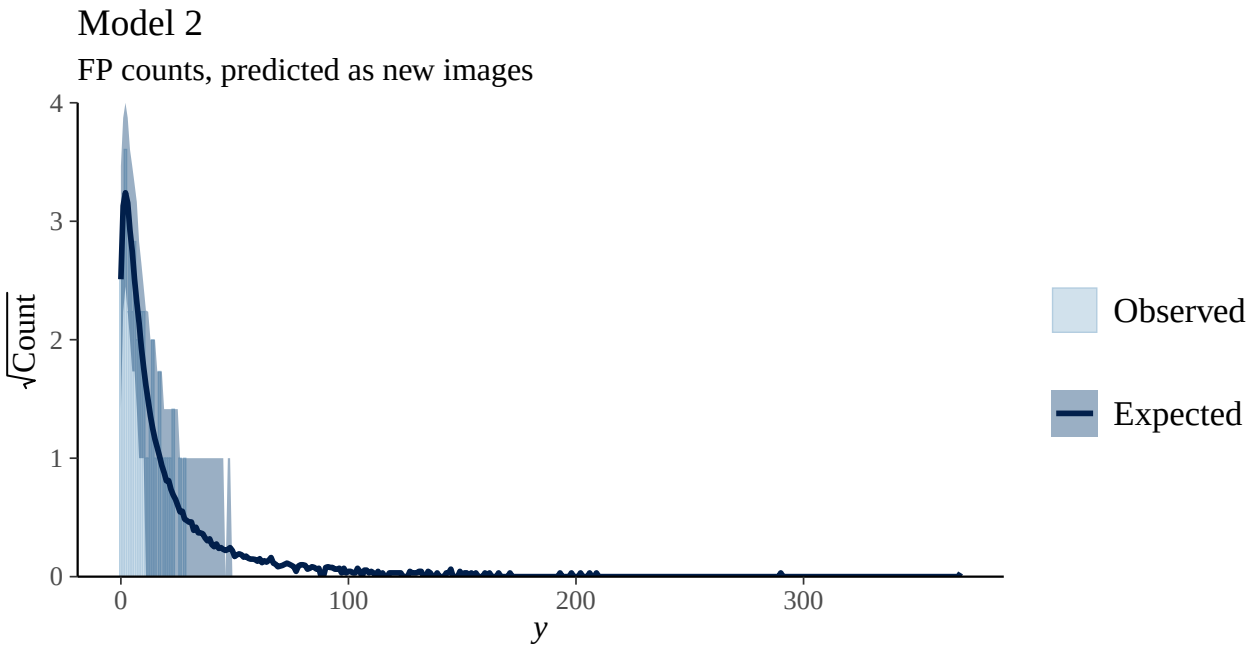



```
# FP counts, predicted as new images
purrr::map2(F1_model_multi, data_list, \(x, y)
  pp_check(x, resp = "FP", type = "rootogram", ndraws = 1000,
           newdata = y |> mutate(Slide = paste0("new_", Slide)),
           allow_new_levels = TRUE, sample_new_levels = "gaussian") +
    ggtitle(unique(y$model), "FP counts, predicted as new images")
)
```

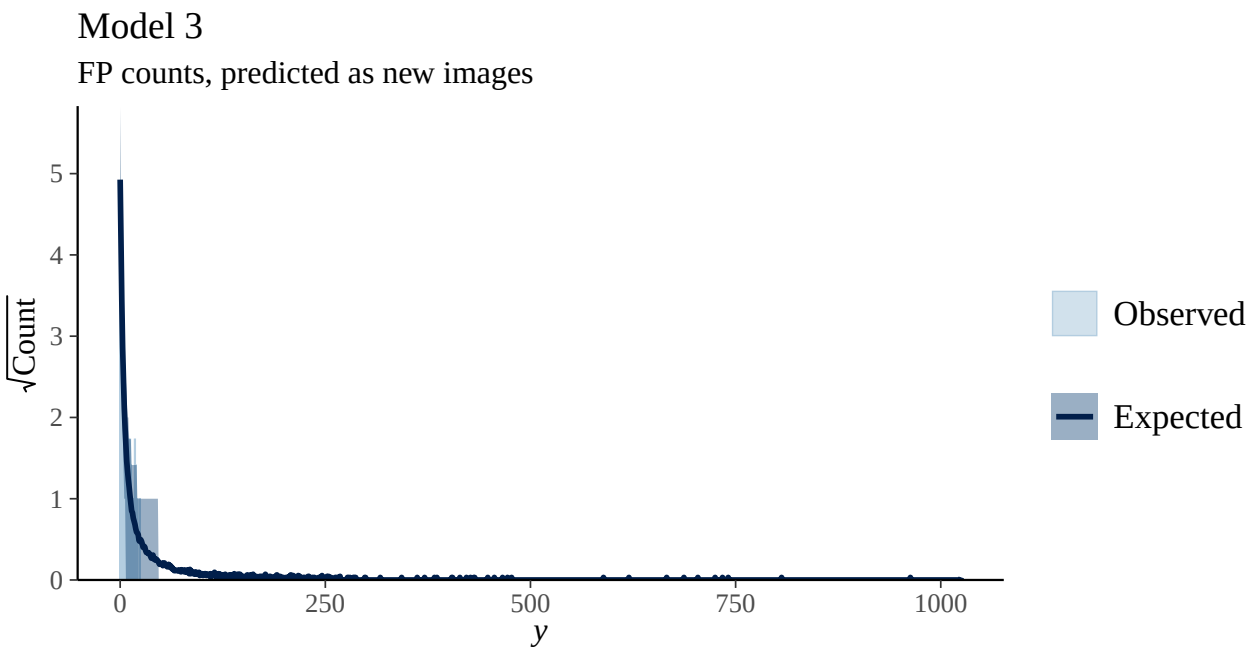
```
[[1]]
```


Model 1
FP counts, predicted as new images
√Count
y
0
100
200
0
1
2
3
4
Observed
Expected

```
[[2]]
```


Model 2
FP counts, predicted as new images
√Count
y
0
100
200
300
0
1
2
3
4
Observed
Expected

```
[[3]]
```



7.2 Checks Part B

7.2.1 Convergence diagnostics

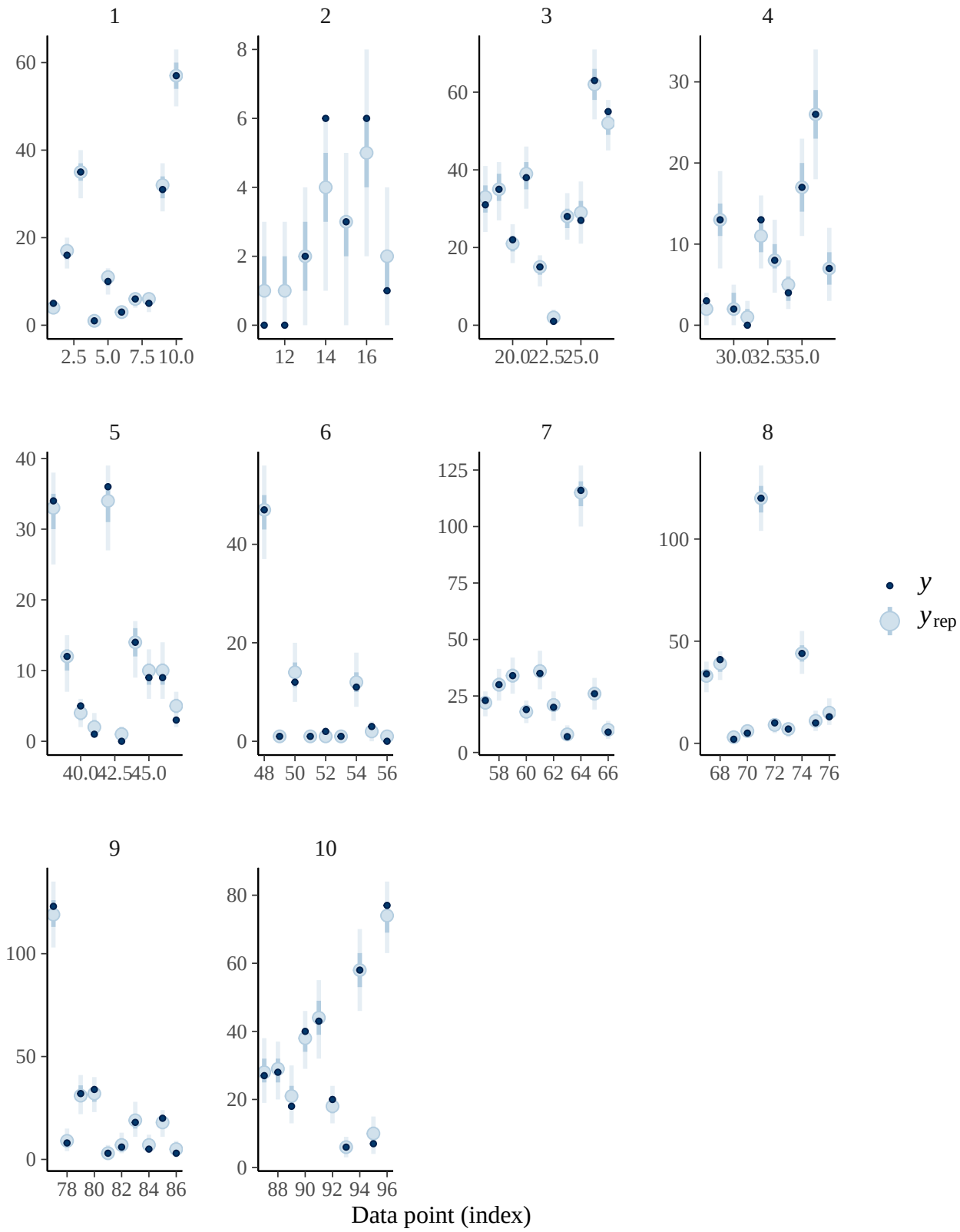
```
# Convergence of the fits of Part B (Domain as fixed effect), same diagnostics
purrr::map2_dfr(F1_model_domain_multi, names(data_list), \(x, y) {
  df_summary <- posterior::summarise_draws(as_draws_array(x), "rhat", "ess_bulk", "ess_tail")
  tibble(
    model        = y,
    max_rhat     = max(df_summary$rhat, na.rm = TRUE),
    min_ess_bulk = min(df_summary$ess_bulk, na.rm = TRUE),
    min_ess_tail = min(df_summary$ess_tail, na.rm = TRUE),
    divergences  = sum(subset(nuts_params(x), Parameter == "divergent__")$Value)
  )
})
```

model	max_rhat	min_ess_bulk	min_ess_tail	divergences
Model 1	1.001124	7431.786	13658.27	0
Model 2	1.000726	7781.160	14806.12	0
Model 3	1.000645	7722.301	12846.27	0

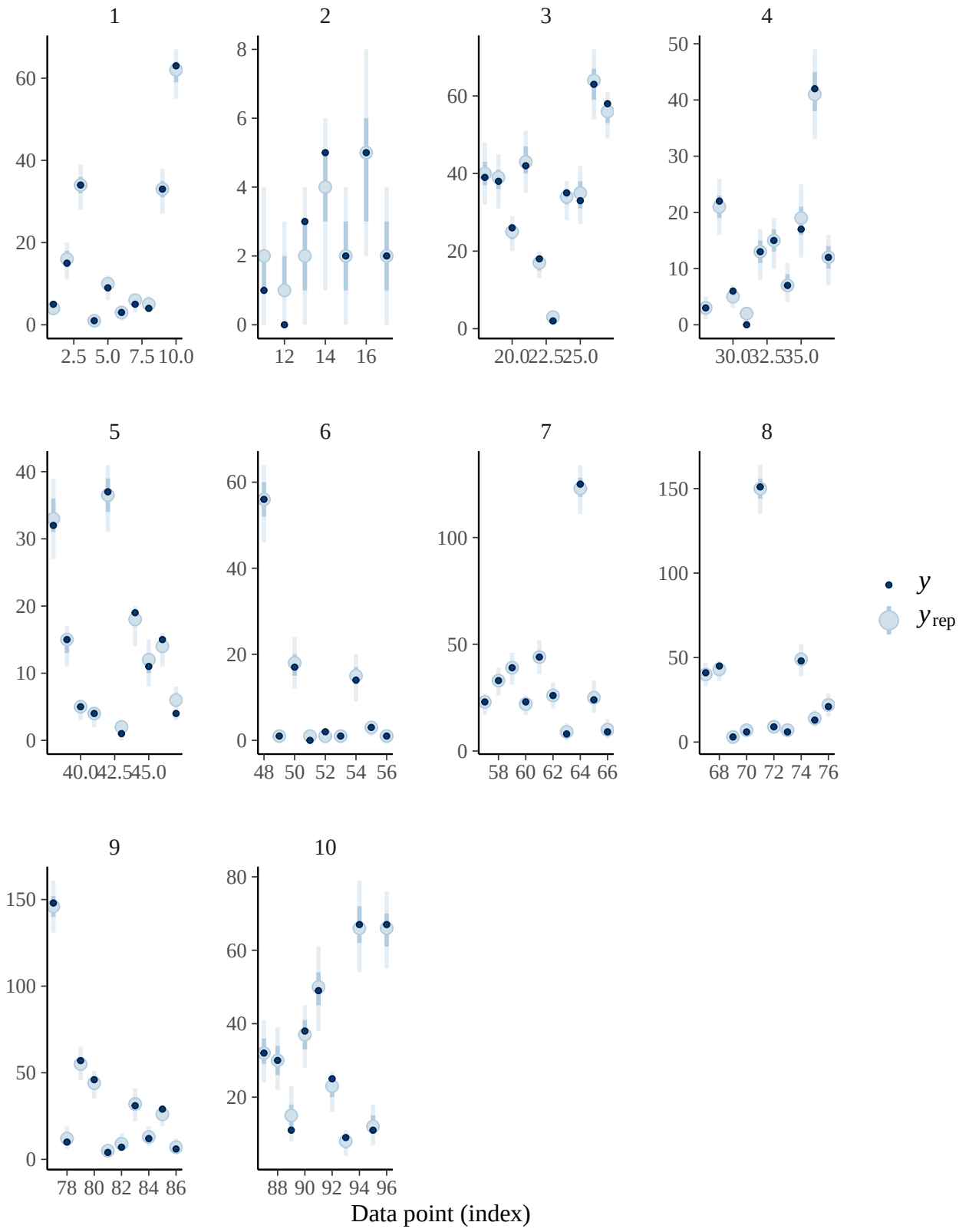
7.2.2 Posterior predictive check

```
# TP: observed TP per image (dot) with the 50% / 90% predictive interval, per domain
purrr::map2(F1_model_domain_multi, names(data_list), \(x, y)
  pp_check(x, resp = "TP", type = "intervals_grouped", group = "Domain", ndraws = 1000) +
    ggtitle(y, "TP per image, by domain")
)
```

```
[[1]]
```


Model 1
TP per image, by domain
1
2
3
4
5
6
7
8
9
10
y
y rep
Data point (index)

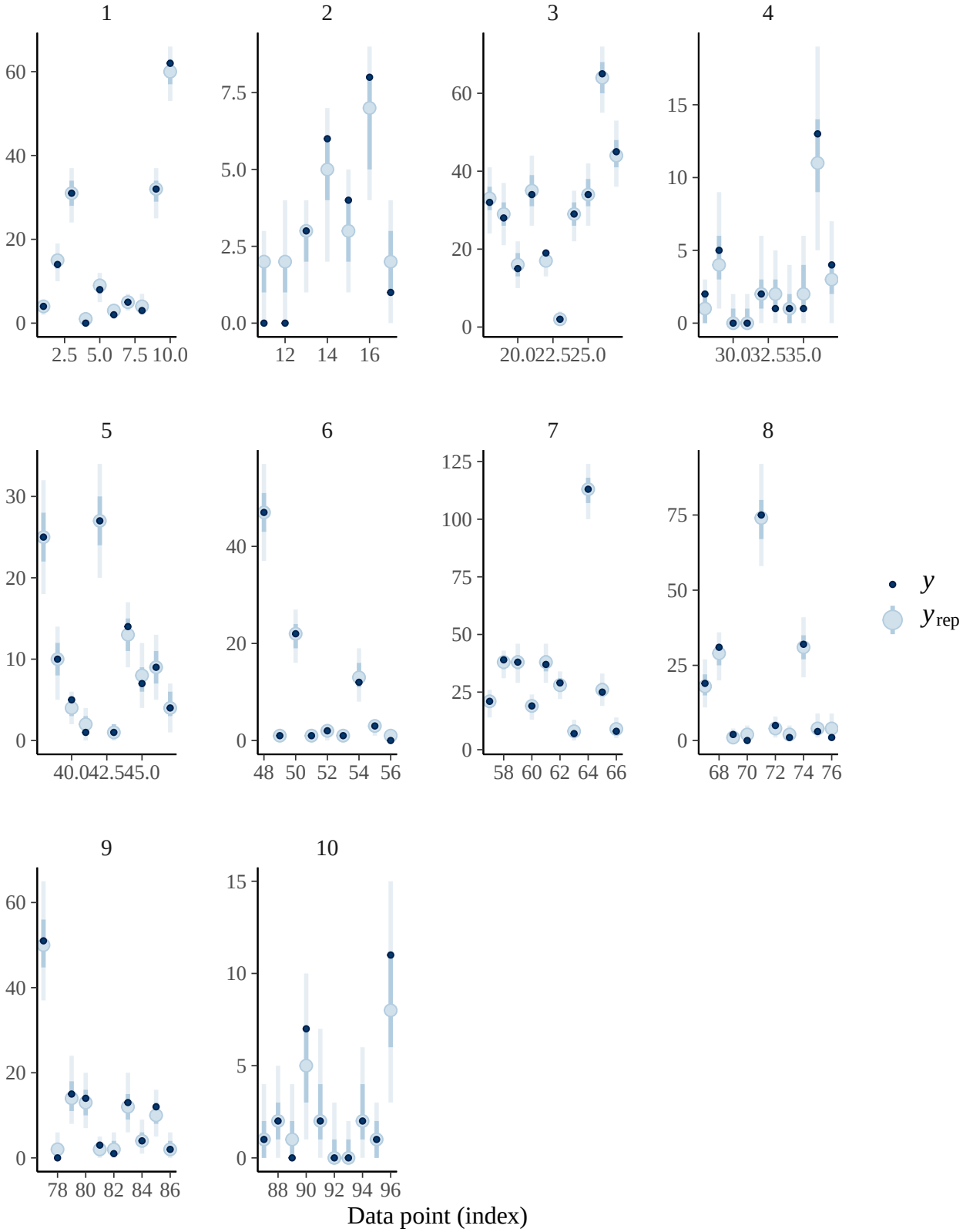
[[2]]


Model 2
TP per image, by domain
1
2
3
4
5
6
7
8
9
10
y
y rep
Data point (index)

[[3]]

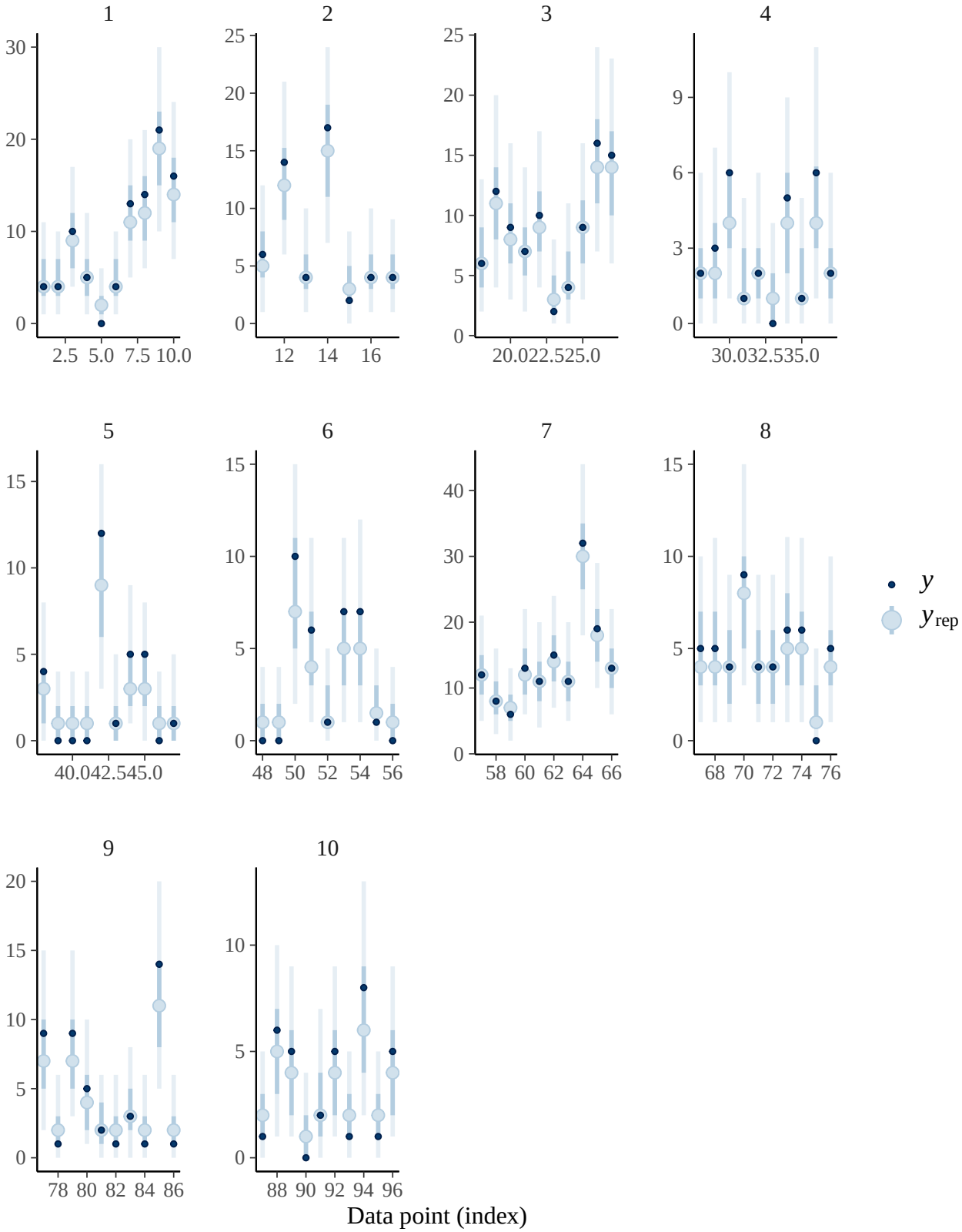
Model 3

TP per image, by domain




```
# FP: rootogram of the FP counts (observed bars vs. expected frequencies)
purrr::map2(F1_model_domain_multi, names(data_list), \(x, y)
  pp_check(x, resp = "FP", type = "intervals_grouped", group = "Domain", ndraws = 1000) +
    ggtitle(y, "FP per image, by domain")
)
```

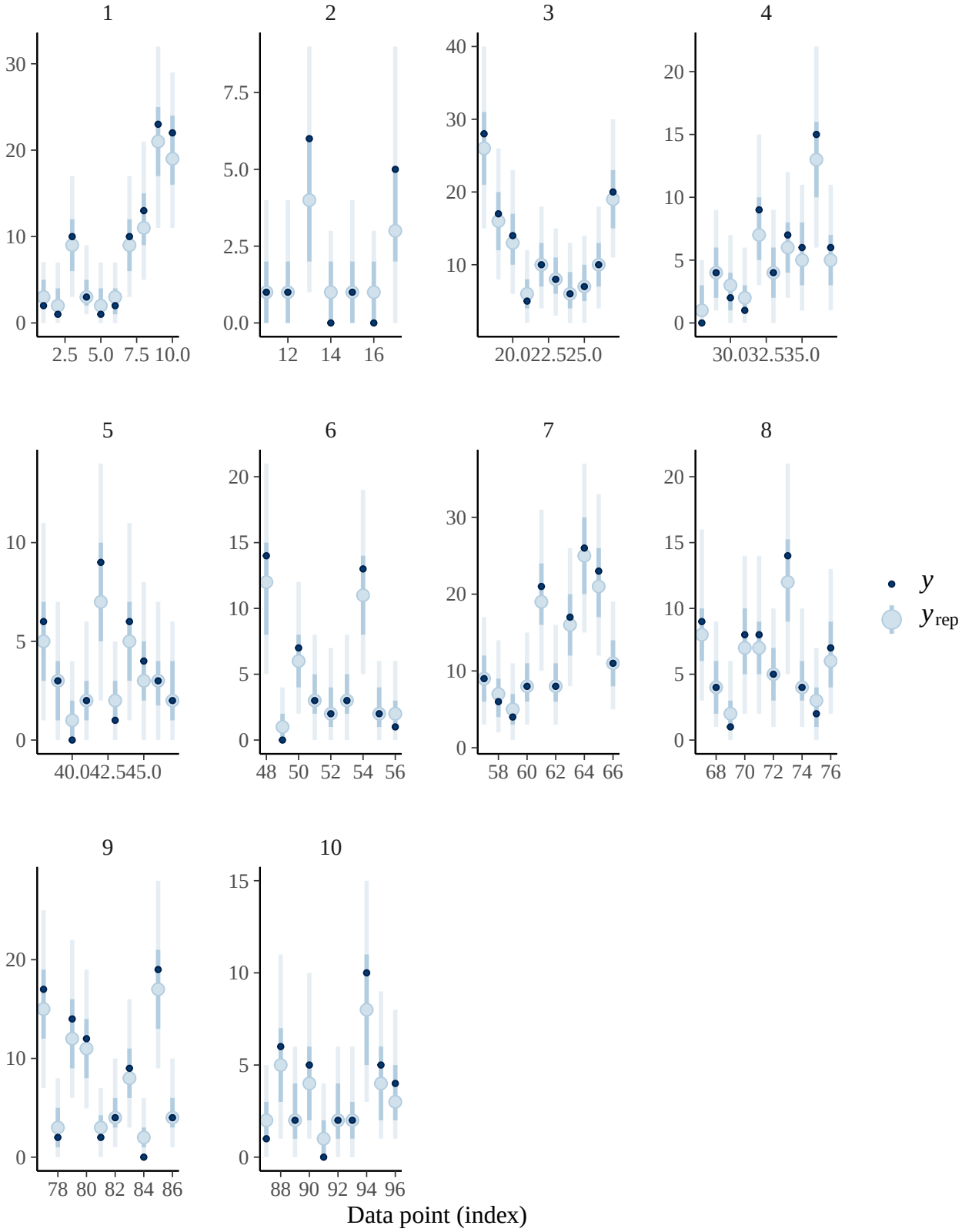
```
[[1]]
```


Model 1
FP per image, by domain
1
2
3
4
5
6
7
8
9
10
y
y rep
Data point (index)

```
[[2]]
```

Model 2

FP per image, by domain



[[3]]

Model 3

FP per image, by domain

